Practical 9

Implement a simple planning problem using STRIPS / goal-stack planning for Blocks World, logistics, or robot navigation.

Objective

To implement a STRIPS-style Blocks World planning problem and solve it using the classical Goal-Stack Planning algorithm, producing a valid ordered sequence of MOVE actions from an initial state to a specified goal state.

Procedure

1. Define the initial state of a 3-block world (A, B, C) in terms of on(x,y), on_table(x) and clear(x) predicates.
2. Define the goal as a conjunction of ON literals: ON(A,B) and ON(B,C), i.e. stack A on B on C.
3. Implement the STRIPS MOVE(block, dest) operator with its preconditions (block clear, destination clear) and effects (updates on/on_table/clear sets).
4. Implement the goal_stack_plan() algorithm: push all goals onto a stack; for each goal popped, if already satisfied skip it, else push any unmet preconditions as sub-goals and retry the goal afterward, otherwise apply the corresponding MOVE action.
5. Trace and print every step of the goal-stack algorithm, including sub-goals pushed and actions applied, along with the resulting world state after each action.
6. Report the final linear action plan and confirm the final state satisfies both goal literals.

Program / Code

```
# ================================================================
# PRACTICAL 9
# STRIPS / Goal-Stack Planning - Blocks World
# ================================================================

# ----------------------------------------------------------------
# Initial state:  C is on A, A and B are on the Table, B is clear, C is clear
#   [C]
#   [A]   [B]
#  -------------- Table
#
# Goal state:  A on B, B on C  (a stack A-on-B-on-C)
#   [A]
#   [B]
#   [C]
#  -------------- Table
# ----------------------------------------------------------------

initial_state = {
    "on": {"C": "A"},          # C is on A
    "on_table": {"A", "B"},    # A and B are on the table
```

```
    "clear": {"B", "C"},       # B and C have nothing on top
}

goal_stack = [
    ("ON", "A", "B"),
    ("ON", "B", "C"),
]


class BlocksWorldState:
    def __init__(self, on, on_table, clear):
        self.on = dict(on)
        self.on_table = set(on_table)
        self.clear = set(clear)

    def clone(self):
        return BlocksWorldState(self.on, self.on_table, self.clear)

    def is_clear(self, block):
        return block in self.clear

    def is_on_table(self, block):
        return block in self.on_table

    def is_on(self, x, y):
        return self.on.get(x) == y

    def describe(self):
        parts = []
        for b, base in self.on.items():
            parts.append(f"{b} on {base}")
        for b in self.on_table:
            parts.append(f"{b} on Table")
        return ", ".join(sorted(parts))


def move(state, block, dest):
    """STRIPS operator: move(block, dest)
    Preconditions: block is clear, dest is clear (or Table), block not already on dest
    Effects: block on dest; old location of block becomes clear;
             dest is no longer clear (unless dest == Table)."""

    plan_step = f"MOVE({block} -> {dest})"

    old_base = state.on.get(block, "Table" if state.is_on_table(block) else None)

    # Remove block from old position
    if block in state.on:
        del state.on[block]
    state.on_table.discard(block)

    # Old base becomes clear again
    if old_base and old_base != "Table":
        state.clear.add(old_base)

    # Place block on new destination
```

```
    if dest == "Table":
        state.on_table.add(block)
    else:
        state.on[block] = dest
        state.clear.discard(dest)

    state.clear.add(block)  # block just placed is clear on top
    return plan_step


def goal_stack_plan(state, goals):
    """Simplified goal-stack planning algorithm."""
    stack = list(goals)   # process goals in order (a stack, LIFO from the back)
    plan = []
    trace = []

    step_no = 0
    while stack:
        goal = stack.pop()
        step_no += 1

        predicate, x, y = goal
        trace.append(f"Step {step_no}: Consider goal {predicate}({x}, {y})")

        if predicate == "ON":
            if state.is_on(x, y):
                trace.append(f"   -> already satisfied: {x} is on {y}")
                continue

            # Preconditions for MOVE(x -> y): x clear, y clear
            sub_goals = []
            if not state.is_clear(y) and y != "Table":
                sub_goals.append(("CLEAR", y, None))
            if not state.is_clear(x):
                sub_goals.append(("CLEAR", x, None))

            if sub_goals:
                trace.append(f"   -> preconditions not met, pushing sub-goals: {sub_goals}")
                stack.append(goal)                 # re-achieve this goal after sub-goals
                stack.extend(reversed(sub_goals))
            else:
                action = move(state, x, y)
                plan.append(action)
                trace.append(f"   -> action applied: {action}")
                trace.append(f"      state now: {state.describe()}")

        elif predicate == "CLEAR":
            if state.is_clear(x):
                trace.append(f"   -> already satisfied: {x} is clear")
                continue
            # Find what's on top of x and move it to the table
            on_top = None
            for b, base in state.on.items():
                if base == x:
                    on_top = b
                    break
            if on_top:
```

```
                action = move(state, on_top, "Table")
                plan.append(action)
                trace.append(f"   -> action applied: {action}")
                trace.append(f"      state now: {state.describe()}")

    return plan, trace


if __name__ == "__main__":
    print("=" * 65)
    print("     STRIPS / GOAL-STACK PLANNING - BLOCKS WORLD")
    print("=" * 65)

    state = BlocksWorldState(**initial_state)

    print("\nInitial State :", state.describe())
    print("Goal Stack    :", goal_stack)
    print("-" * 65)

    plan, trace = goal_stack_plan(state, goal_stack)

    print("\nPlanning Trace:")
    for line in trace:
        print(line)

    print("-" * 65)
    print("\nFinal Plan (sequence of actions):")
    for i, action in enumerate(plan, start=1):
        print(f"  {i}. {action}")

    print("\nFinal State :", state.describe())
    print("=" * 65)
```

Output

```
=================================================================
     STRIPS / GOAL-STACK PLANNING - BLOCKS WORLD
=================================================================

Initial State : A on Table, B on Table, C on A
Goal Stack    : [('ON', 'A', 'B'), ('ON', 'B', 'C')]
-----------------------------------------------------------------

Planning Trace:
Step 1: Consider goal ON(B, C)
   -> action applied: MOVE(B -> C)
      state now: A on Table, B on C, C on A
Step 2: Consider goal ON(A, B)
   -> preconditions not met, pushing sub-goals: [('CLEAR', 'A', None)]
Step 3: Consider goal CLEAR(A, None)
   -> action applied: MOVE(C -> Table)
      state now: A on Table, B on C, C on Table
Step 4: Consider goal ON(A, B)
   -> action applied: MOVE(A -> B)
      state now: A on B, B on C, C on Table
```

```
------------------------------------------------------------------

Final Plan (sequence of actions):
  1. MOVE(B -> C)
  2. MOVE(C -> Table)
  3. MOVE(A -> B)

Final State : A on B, B on C, C on Table
==================================================================
```

Conclusion

The Goal-Stack Planning algorithm produced a valid 3-step STRIPS plan — MOVE(B->C), MOVE(C->Table), MOVE(A->B) — that transformed the initial state (C on A; A, B on the table) into the goal state (A on B on C on the table). The trace shows the algorithm correctly detecting an unmet precondition (block A was not clear because C sat on it) and dynamically pushing a CLEAR(A) sub-goal onto the stack before re-attempting ON(A,B), which is exactly the recursive means-ends behaviour that makes goal-stack planning work. The experiment demonstrates how a STRIPS action representation (explicit preconditions and add/delete effects) combined with a simple stack-based control strategy is sufficient to automatically sequence actions for classical, fully-observable, deterministic planning problems like Blocks World.